Imputation du débit par les rues voisines

Comment fonctionne `voisins_imputation.ipynb`

1. Le problème et l'idée

Nos trois arcs (Champs-Élysées, Convention, Saints-Pères) ont des heures **sans mesure** : capteur en panne, arc invalide, fichiers manquants... Ces trous appauvrissent l'historique sur lequel on entraîne le modèle de prévision.

L'idée de Rim : les rues d'à côté sont mesurées par d'autres capteurs, souvent au même moment. Leur trafic monte et descend avec le nôtre (mêmes heures de pointe, mêmes jours fériés, mêmes bouchons). Si on apprend la relation *débit voisins -> débit de notre arc* sur les heures où tout est mesuré, on peut l'appliquer aux heures où seul notre arc manque.

Axe cible	Rues voisines utilisées
Champs-Élysées	Av_des_Champs_Elysees (autres arcs), Tilsitt, Presbourg, Av_Marceau
Convention	Convention (autres arcs), Lecourbe, Vaugirard
Saints-Pères	Sts_Peres (autres arcs), Quai_Voltaire, Quai_Malaquais, Bd_St_Germain

Une rue = plusieurs **capteurs** (un par arc, c'est-à-dire un tronçon entre deux carrefours, dans un sens). C'est chaque capteur qui devient une variable, pas la rue entière.

2. Les 4 étapes du notebook

```
archives .txt 2014-2025  +  API open data (oct. 2025 -> auj.)
            |
   [1] table large : 1 ligne = 1 heure UTC, 1 colonne = 1 capteur voisin
            |
   [2] par axe : on garde les capteurs avec corrélation >= 0.8
            |
   [3] modèle boosting  (voisins + heure + jour)  ->  validation par semaines
            |
   [4] on prédit les heures où notre arc manque mais un voisin est mesuré
```

Étape 1 - Récupérer les débits des voisins

- **2014 -> sept. 2025** : nos archives locales `data/unprocessed_data/<année>/*.txt` contiennent *tous* les capteurs de Paris. On lit seulement 4 colonnes (arc, libellé, heure, débit) et on filtre sur les noms de rues. Lecture en parallèle (8 threads).
- **oct. 2025 -> aujourd'hui** : nos exports 2026 ne contiennent que nos 3 arcs, donc on interroge l'API open data de la Ville de Paris (dataset *comptages-routiers-permanents*), filtrée sur les mêmes rues.
- Heures : dans les archives, `t_1h` est en heure de Paris -> converti en UTC exactement comme dans `dataset_create.py`, pour que les heures tombent pile sur celles de notre cible.
- Résultat pivoté en table large (`rue_arcid` en colonnes) et mis en cache dans `data/external/voisins.csv` : la longue extraction (~10 min) n'a lieu qu'une fois.

Étape 2 - Choisir les bons capteurs

Pour chaque axe, on calcule la **corrélation de Pearson** entre chaque capteur voisin et notre débit, sur les heures où les deux sont mesurés (au moins 500 heures communes, sinon la corrélation n'est pas fiable). On garde ceux à **0.8 ou plus** (même seuil que Rim). Un capteur sur une rue parallèle à sens unique inverse, par exemple, sera peu corrélé et écarté.

Attention à la fuite : certains « voisins » sont en fait des **copies** de notre arc (`Convention_5674`, `Sts_Peres_192` : 100 % de valeurs identiques). Ils tombent en panne en même temps que lui, donc ne servent à rien pour combler les trous, mais le modèle s'appuie dessus et la validation devient absurde (MAE de 0,3 %). C'est ce qui explique le RMSE de 4e-13 de Rim sur Convention. Règle ajoutée : un capteur n'est gardé que s'il est **mesuré pendant au moins 24 heures où notre arc manque**. Cela élimine l'arc cible et ses copies automatiquement.

Résultat : 27 capteurs retenus pour Champs-Élysées, 19 pour Saints-Pères ; pour Convention, essentiellement Lecourbe, Vaugirard et les autres arcs de Convention.

Étape 3 - Le modèle et sa validation

Variables d'entrée : les débits des capteurs retenus + `heure` et `jour_sem` (heure de Paris), car la relation voisin/cible peut changer selon le moment. Modèle : `HistGradientBoostingRegressor` (scikit-learn, même famille que LightGBM).

- Il **gère les NaN tout seul** : si un voisin est aussi en panne à cette heure-là, le modèle fait avec les autres. Rim devait remplacer les trous par la médiane (`SimpleImputer`), ce qui injecte de fausses valeurs.
- Il capte les relations non linéaires (ex. un voisin saturé ne transmet plus d'information).
- Les prédictions négatives sont ramenées à 0 (`clip(0)`).

Validation par semaines entières. Rim masquait 10 % des heures au hasard. Problème : pour une heure masquée, l'heure d'avant et celle d'après restent dans le train, et le trafic change peu d'une heure à l'autre, donc le score est trop optimiste. Ici on masque **20 % des semaines complètes**, ce qui ressemble beaucoup plus à un vrai trou (une panne dure souvent des jours). Métriques : RMSE, MAE et MAE en % du débit moyen.

Étape 4 - Combler les trous

On réentraîne le modèle sur toutes les heures observées, puis on prédit les heures où **notre débit manque ET au moins un capteur retenu est mesuré**. Les heures déjà observées ne sont jamais modifiées. Le notebook affiche le % de débit manquant par année, avant et après, et écrit `data/processed_data/debit_voisins.csv` (axe, datetime_utc, debit_voisins).

Résultats

Axe	MAE	MAE / débit moyen	RMSE
Champs-Élysées	37,6	3,7 %	79,8
Convention	39,8	8,1 %	60,0
Saints-Pères	42,6	9,5 %	71,8

Validation sur 20 % de semaines masquées, en véhicules par heure. Champs-Élysées est sans doute un peu optimiste : l'arc 4274 y est quasi identique au nôtre (97,5 % des heures), mais reste mesuré pendant ~6 300 heures de trous, donc il est légitime. Il reste une estimation de 4 à 10 %, à comparer aux ~12 % de Rim.

% de débit manquant	2019	2022	2023	2024	2025	2026
Champs-Élysées	27,5 -> 0,7	14,3 -> 3,2	1,4 -> 0,4	9,9 -> 6,4	18,1 -> 1,1	**100 -> 3,0**
Convention	1,0 -> 0,7	19,1 -> 3,2	58,0 -> 0,3	71,9 -> 6,3	35,0 -> 0,5	3,6 -> 1,8

Saints-Pères	13,9 -> 0,7	4,0 -> 3,2	1,8 -> 1,2	45,9 -> 36,9	80,8 -> 1,0	2,2 -> 1,8

Point clé : en 2026, l'arc des Champs-Élysées est entièrement marqué « Invalide » (100 % manquant). Sans les voisins, on n'a aucune donnée récente pour l'arc qu'on doit prédire.

3. Ce que ça change par rapport à la version de Rim

Point	Rim	Notebook
Données voisines	Exports récents (~13 mois)	Archives 2014-2025 + API : tout l'historique
Voisins manquants	Remplis par la médiane	NaN gérés nativement
Modèle	Régression linéaire vs random forest	Gradient boosting + heure/jour
Validation	10 % d'heures au hasard	20 % de semaines entières
Arc cible	Fuite possible (Convention)	Copies exclues (règle « mesuré pendant les trous »)
Code	~450 lignes	~80 lignes

4. Limites à garder en tête

- **Pannes globales** : quand tout le réseau de capteurs tombe (fichiers manquants), les voisins manquent aussi, donc ces heures restent vides.
- **Pas utilisable pour la prévision du 7 au 11 novembre** : à ce moment-là on n'a pas non plus les débits des voisins. Cette méthode sert à *enrichir l'historique d'entraînement*, pas à fournir des features au modèle final.
- Une valeur imputée reste une estimation : garder une colonne « imputé oui/non » pour pouvoir lui donner moins de poids à l'entraînement, ou comparer le modèle avec et sans.
- Le seuil de 0.8 et la liste des rues se règlent en haut du notebook (`SEUIL_CORR`, `VOISINS`).